

PROJET D'EXAMEN FINAL

Optimisation pour le Machine Learning

Master SSD 2025-2026

FST-UN

Janvier 2026

Table des matières

1	Modélisation et Étude Théorique	2
2	Stochasticité et Passage à l'Échelle	5
3	Parcimonie et Algorithmes Proximaux	8

1 Modélisation et Étude Théorique

a) Modélisation du problème

Pour le dataset YearPredictionMSD avec $n \approx 515\,000$ exemples et $d = 90$ caractéristiques, nous cherchons à prédire l'année de sortie d'une chanson. Soit $X \in \mathbb{R}^{n \times d}$ la matrice des données et $y \in \mathbb{R}^n$ le vecteur cible.

Formulation du problème

Le problème de régression linéaire ridge s'écrit sous la forme d'une somme finie :

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w) + \frac{\mu}{2} \|w\|^2 \quad (1)$$

où chaque fonction $f_i(w)$ représente l'erreur quadratique pour l'exemple i :

$$f_i(w) = \frac{1}{2} (x_i^T w - y_i)^2$$

avec $x_i \in \mathbb{R}^d$ le i -ème vecteur de caractéristiques et $y_i \in \mathbb{R}$ la cible correspondante.

La fonction objectif complète devient :

$$f(w) = \frac{1}{2n} \sum_{i=1}^n (x_i^T w - y_i)^2 + \frac{\mu}{2} \|w\|^2 = \frac{1}{2n} \|Xw - y\|^2 + \frac{\mu}{2} \|w\|^2$$

Justification de la pénalité ridge

L'ajout du terme de régularisation $\frac{\mu}{2} \|w\|^2$ avec $\mu > 0$ garantit l'unicité du minimum global selon le **Théorème 1.2.9** du cours :

Théorème 1 (Théorème 1.2.9). *Si $f : \mathbb{R}^d \rightarrow \mathbb{R}$ est fortement convexe, elle possède un unique minimum global. De plus, si f est de classe $\mathcal{C}^1$, ce minimum est l'unique solution de $\nabla f(w) = 0$.*

Dans notre cas :

La fonction $f_i(w) = \frac{1}{2} (x_i^T w - y_i)^2$ est convexe car c'est une fonction quadratique. La somme de fonctions convexes est convexe. Le terme $\frac{\mu}{2} \|w\|^2$ est μ -fortement convexe.

Pour tout $w, v \in \mathbb{R}^d$ et $\lambda \in [0, 1]$:

$$\frac{\mu}{2} \|\lambda w + (1 - \lambda)v\|^2 \leq \lambda \frac{\mu}{2} \|w\|^2 + (1 - \lambda) \frac{\mu}{2} \|v\|^2 - \frac{\mu}{2} \lambda(1 - \lambda) \|w - v\|^2$$

Par conséquent, $f(w)$ hérite de la forte convexité avec paramètre μ , garantissant l'unicité du minimum global même si X n'est pas de rang plein.

b) Calcul du gradient et de la Hessienne

Calculons $\nabla f(w)$ analytiquement :

$$\begin{aligned}\nabla f(w) &= \nabla \left[\frac{1}{2n} \|Xw - y\|^2 + \frac{\mu}{2} \|w\|^2 \right] \\ &= \frac{1}{2n} \nabla [(Xw - y)^T (Xw - y)] + \frac{\mu}{2} \nabla (w^T w) \\ &= \frac{1}{2n} \nabla [w^T X^T X w - 2y^T X w + y^T y] + \mu w \\ &= \frac{1}{2n} (2X^T X w - 2X^T y) + \mu w \\ &= \frac{1}{n} X^T (Xw - y) + \mu w\end{aligned}$$

Donc :

$$\boxed{\nabla f(w) = \frac{1}{n} X^T (Xw - y) + \mu w}$$

La matrice Hessienne est la dérivée seconde de f :

$$\begin{aligned}\nabla^2 f(w) &= \nabla \left[\frac{1}{n} X^T X w - \frac{1}{n} X^T y + \mu w \right] \\ &= \frac{1}{n} X^T X + \mu I_d\end{aligned}$$

Donc :

$$\boxed{\nabla^2 f(w) = \frac{1}{n} X^T X + \mu I_d}$$

Cette Hessienne est constante (indépendante de w), ce qui caractérise une fonction quadratique.

c) Constante de Lipschitz via SVD

La décomposition SVD de X s'écrit :

$$X = U \Sigma V^T$$

où :

- $U \in \mathbb{R}^{n \times n}$ est orthogonale ($U^T U = I_n$)
- $V \in \mathbb{R}^{d \times d}$ est orthogonale ($V^T V = I_d$)
- $\Sigma \in \mathbb{R}^{n \times d}$ est diagonale avec les valeurs singulières $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(n,d)} \geq 0$

Calcul de la constante de Lipschitz

Le gradient ∇f est L-Lipschitz si :

$$\|\nabla f(w) - \nabla f(v)\| \leq L \|w - v\|, \quad \forall w, v \in \mathbb{R}^d$$

Pour une fonction deux fois différentiable, la constante de Lipschitz du gradient est la plus grande valeur propre de la Hessienne :

$$L = \lambda_{\max}(\nabla^2 f(w))$$

Or, $\nabla^2 f(w) = \frac{1}{n}X^T X + \mu I_d$. Les valeurs propres de $X^T X$ sont les carrés des valeurs singulières de X :

$$\lambda_i(X^T X) = \sigma_i^2(X), \quad i = 1, \dots, d$$

Par conséquent :

$$\begin{aligned} \lambda_{\max}(\nabla^2 f(w)) &= \lambda_{\max}\left(\frac{1}{n}X^T X + \mu I_d\right) \\ &= \frac{1}{n}\lambda_{\max}(X^T X) + \mu \\ &= \frac{1}{n}\sigma_{\max}^2(X) + \mu \end{aligned}$$

Donc :

$$L = \frac{\sigma_{\max}^2(X)}{n} + \mu$$

où $\sigma_{\max}(X)$ est la plus grande valeur singulière de X .

Impact sur la stabilité

La constante de Lipschitz L joue un rôle crucial dans la stabilité de la descente de gradient :

1. **Condition de convergence :** Pour que la descente de gradient avec pas constant α converge, il faut :

$$\alpha < \frac{2}{L}$$

Un pas optimal est souvent $\alpha = \frac{1}{L}$.

2. **Taux de convergence :** Pour une fonction μ -fortement convexe avec gradient L -Lipschitz, le taux de convergence de la descente de gradient est :

$$\|w_k - w^*\| \leq \left(1 - \frac{\mu}{L}\right)^k \|w_0 - w^*\|$$

Le ratio $\kappa = \frac{L}{\mu}$ est le *nombre de condition*. Un grand κ implique une convergence plus lente.

3. **Stabilité numérique :** Une grande valeur de L (due à $\sigma_{\max}$ élevé) nécessite un pas d'apprentissage très petit, ralentissant la convergence. La standardisation des données réduit $\sigma_{\max}$, améliorant la stabilité.

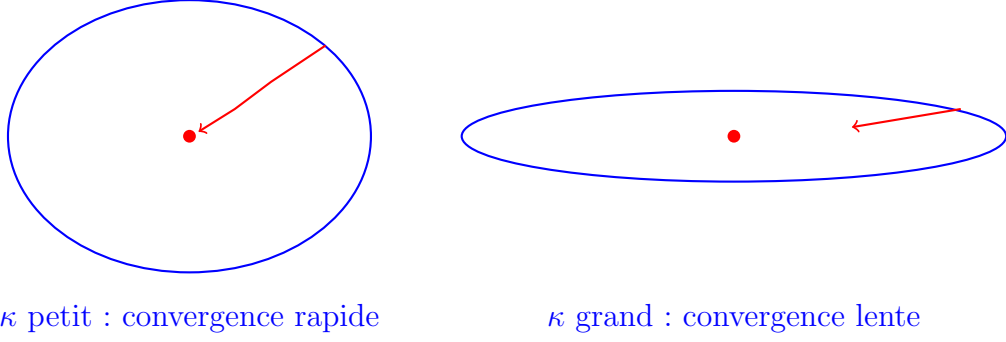


FIGURE 1 – Impact du nombre de condition sur la convergence de la descente de gradient

2 Stochasticité et Passage à l'Échelle

a) Descente de Gradient Stochastique (SGD)

Principe du SGD

Avec $n > 500\,000$ exemples, calculer le gradient complet $\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w)$ à chaque itération est prohibitif en termes de temps de calcul. Le SGD sélectionne aléatoirement un seul exemple i_k à l'itération k et utilise :

$$g_k = \nabla f_{i_k}(w_k) = x_{i_k}(x_{i_k}^T w_k - y_{i_k}) + \mu w_k$$

La mise à jour devient :

$$w_{k+1} = w_k - \alpha_k g_k$$

Théorème 2. *L'estimateur stochastique $g_k = \nabla f_{i_k}(w_k)$ est sans biais pour le gradient complet :*

$$\mathbb{E}[g_k] = \nabla f(w_k)$$

Démonstration. Si l'indice i_k est choisi uniformément dans $\{1, 2, \dots, n\}$, alors :

$$\begin{aligned}
 \mathbb{E}[g_k] &= \mathbb{E}[\nabla f_{i_k}(w_k)] \\
 &= \sum_{i=1}^n \mathbb{P}(i_k = i) \cdot \nabla f_i(w_k) \\
 &= \sum_{i=1}^n \frac{1}{n} \nabla f_i(w_k) \\
 &= \frac{1}{n} \sum_{i=1}^n \nabla f_i(w_k) \\
 &= \nabla f(w_k)
 \end{aligned}$$

Ceci prouve que g_k est un estimateur sans biais du gradient complet. □

Algorithme SGD

Algorithm 1 Descente de Gradient Stochastique (SGD)

Require: $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$, $\mu > 0$, T (nombre d'itérations), α_0 (pas initial)

- 1: Initialiser $w_0 \in \mathbb{R}^d$ (par exemple, $w_0 = 0$)
 - 2: **for** $k = 0, 1, \dots, T - 1$ **do**
 - 3: Choisir uniformément $i_k \in \{1, \dots, n\}$
 - 4: Calculer le gradient stochastique : $g_k = x_{i_k}(x_{i_k}^T w_k - y_{i_k}) + \mu w_k$
 - 5: Calculer le pas d'apprentissage : $\alpha_k = \frac{\alpha_0}{1+k/n}$ (décroissance)
 - 6: Mettre à jour : $w_{k+1} = w_k - \alpha_k g_k$
 - 7: **end for**
 - 8: **return** w_T
-

b) Analyse Comparative : Batch vs SGD

Gradient de Batch (Mini-batch complet)

Pour un sous-échantillon de taille $B \ll n$:

$$g_k^{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}_k} \nabla f_i(w_k)$$

Phénomène de bruit de gradient

Le SGD introduit un bruit stochastique dans les mises à jour :

$$\text{Var}(g_k) = \mathbb{E}[\|g_k - \nabla f(w_k)\|^2]$$

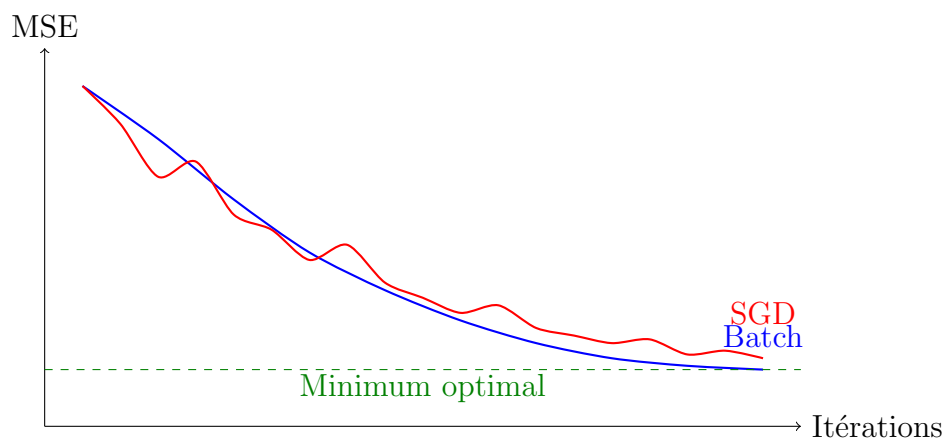


FIGURE 2 – Convergence : Batch (lisse) vs SGD (avec bruit)

c) Mini-batch et Adam

Mini-batch SGD

Le mini-batch combine les avantages du batch et du SGD :

$$w_{k+1} = w_k - \alpha_k \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w_k)$$

Compromis variance-temps :

- Batch size petit : plus de bruit, plus d'itérations/seconde
- Batch size grand : moins de bruit, moins d'itérations/seconde

Algorithme Adam (Adaptive Moment Estimation)

Adam combine les moments du premier et second ordre avec correction de biais :

Algorithm 2 Adam

Require: $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$

- 1: Initialiser $w_0, m_0 = 0, v_0 = 0$
 - 2: **for** $k = 0, 1, \dots, T - 1$ **do**
 - 3: Calculer le gradient (mini-batch) : g_k
 - 4: Mettre à jour le moment d'ordre 1 : $m_{k+1} = \beta_1 m_k + (1 - \beta_1) g_k$
 - 5: Mettre à jour le moment d'ordre 2 : $v_{k+1} = \beta_2 v_k + (1 - \beta_2) g_k^2$
 - 6: Corriger le biais : $\hat{m}_{k+1} = \frac{m_{k+1}}{1 - \beta_1^{k+1}}$
 - 7: Corriger le biais : $\hat{v}_{k+1} = \frac{v_{k+1}}{1 - \beta_2^{k+1}}$
 - 8: Mettre à jour : $w_{k+1} = w_k - \alpha \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1} + \epsilon}}$
 - 9: **end for**
-

Avantages d'Adam :

- Pas d'apprentissage adaptatif par paramètre
- Robuste aux échelles différentes des gradients
- Moins sensible au choix de α initial
- Convergence plus rapide en pratique

Importance de la standardisation

La standardisation des données transforme X en $\tilde{X}$:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

où μ_j et σ_j sont la moyenne et l'écart-type de la j -ème colonne.

Impact sur la Hessienne :

La standardisation réduit le nombre de condition de la Hessienne. Si les features ont des échelles très différentes, $\nabla^2 f(w)$ est mal conditionnée :

$$\kappa(\nabla^2 f) = \frac{\lambda_{\max}}{\lambda_{\min}} \gg 1$$

Après standardisation, les valeurs singulières de $\tilde{X}$ sont plus équilibrées, donc :

$$\kappa(\nabla^2 \tilde{f}) \approx 1$$

Ceci améliore drastiquement la convergence des algorithmes d'optimisation.

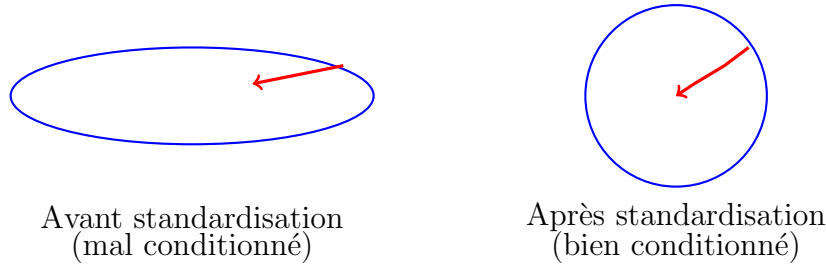


FIGURE 3 – Effet de la standardisation sur les courbes de niveau et la trajectoire d'optimisation

3 Parcimonie et Algorithmes Proximaux

a) Analyse Géométrique : L1 vs L2

Régularisation L2 (Ridge)

La régularisation L2 pénalise :

$$f(w) = \text{Loss}(w) + \frac{\lambda}{2} \|w\|_2^2 = \text{Loss}(w) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2$$

Propriétés géométriques :

- Contrainte : $\|w\|_2^2 \leq t$ (boule euclidienne)
- Différentiable partout
- Rétrécit tous les coefficients proportionnellement
- Ne produit **pas** de parcimonie (coefficients petits mais non nuls)

Régularisation L1 (Lasso)

La régularisation L1 pénalise :

$$f(w) = \text{Loss}(w) + \lambda \|w\|_1 = \text{Loss}(w) + \lambda \sum_{j=1}^d |w_j|$$

Propriétés géométriques :

- Contrainte : $\|w\|_1 \leq t$ (boule ℓ_1 , losange en 2D)
- Non différentiable en $w_j = 0$
- Produit des solutions **parcimonieuses** (beaucoup de $w_j = 0$)
- Sélection automatique de variables

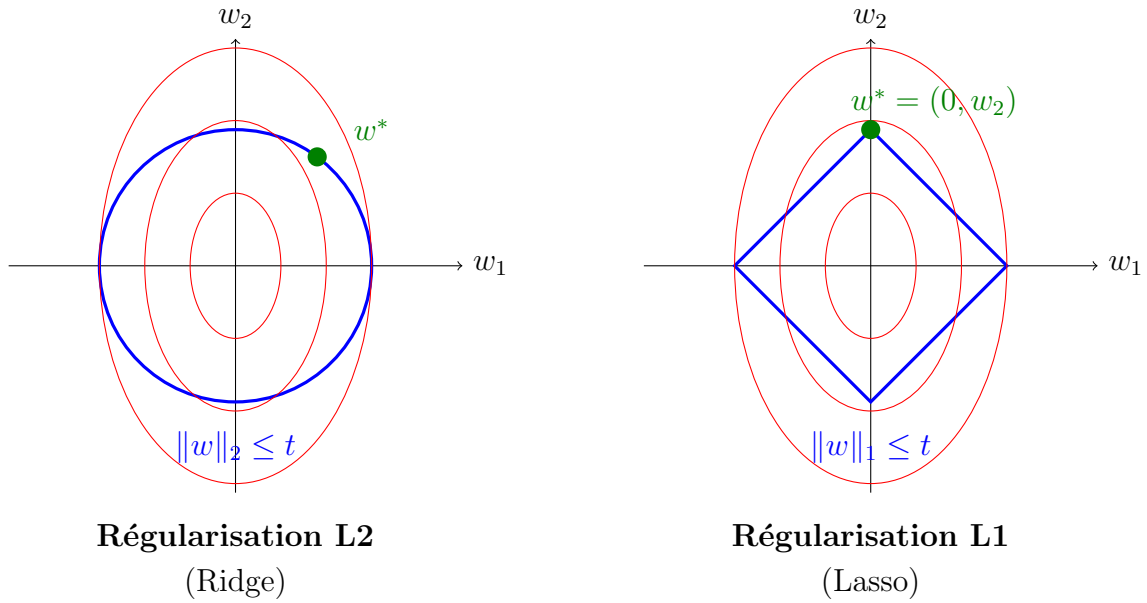


FIGURE 4 – Comparaison géométrique L1 vs L2 : La boule L1 a des coins aux axes, favorisant les solutions avec $w_j = 0$

Pourquoi L1 favorise la parcimonie ?

Argument géométrique :

Le problème contraint équivalent est :

$$\min_w \text{Loss}(w) \quad \text{s.t.} \quad \|w\|_p \leq t$$

La solution optimale est au point de contact entre les courbes de niveau de la Loss et la boule ℓ_p .

- **L2** : La boule euclidienne est lisse, le contact se fait rarement sur les axes $\Rightarrow$ coefficients petits mais non nuls.
- **L1** : La boule ℓ_1 a des **coins pointus** sur les axes. Les courbes de niveau rencontrent souvent ces coins $\Rightarrow$ plusieurs $w_j = 0$.

Haute dimension :

En dimension $d \gg 1$ (comme Reuters RCV1), la probabilité que la solution optimale soit sur un coin (donc parcimonieuse) augmente exponentiellement. La régularisation L1 est donc particulièrement adaptée pour :

- Sélection de variables automatique
- Interprétabilité (peu de features actives)
- Réduction de la complexité du modèle
- Éviter le sur-apprentissage en haute dimension

b) Algorithme ISTA

Opérateur proximal de la norme L1

L'opérateur proximal (ou soft-thresholding) de $\lambda \|\cdot\|_1$ est défini par :

$$\text{prox}_{\lambda \|\cdot\|_1}(v) = \arg \min_w \left\{ \frac{1}{2} \|w - v\|^2 + \lambda \|w\|_1 \right\}$$

Solution analytique composante par composante :

Théorème 3 (Soft-thresholding). *Pour tout $v \in \mathbb{R}^d$:*

$$[\text{prox}_{\lambda\|\cdot\|_1}(v)]_j = S_\lambda(v_j) = \text{sign}(v_j) \max(|v_j| - \lambda, 0)$$

où S_λ est l'opérateur de seuillage doux (soft-thresholding).

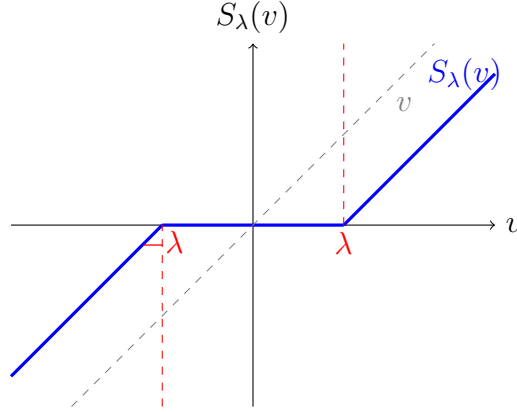


FIGURE 5 – Opérateur de soft-thresholding : valeurs entre $-\lambda$ et λ sont mises à zéro

Algorithme ISTA

ISTA (Iterative Soft-Thresholding Algorithm) résout :

$$\min_w \{f(w) + \lambda\|w\|_1\}$$

où $f(w) = \frac{1}{2n}\|Xw - y\|^2$ est lisse avec gradient L -Lipschitz.

Algorithm 3 ISTA (Iterative Soft-Thresholding Algorithm)

Require: X, y, λ, L (constante de Lipschitz), T (itérations)

- 1: Initialiser $w_0 \in \mathbb{R}^d$
 - 2: Choisir le pas $\alpha = \frac{1}{L}$
 - 3: **for** $k = 0, 1, \dots, T - 1$ **do**
 - 4: Calcul du gradient : $g_k = \frac{1}{n}X^T(Xw_k - y)$
 - 5: Descente de gradient : $v_{k+1} = w_k - \alpha g_k$
 - 6: Soft-thresholding : $w_{k+1} = S_{\alpha\lambda}(v_{k+1})$
 - 7: $w_{k+1,j} = \text{sign}(v_{k+1,j}) \max(|v_{k+1,j}| - \alpha\lambda, 0)$ pour tout j
 - 8: **end for**
 - 9: **return** w_T
-

c) Algorithme accéléré FISTA

Principe de l'accélération de Nesterov

FISTA (Fast ISTA) utilise le schéma d'accélération de Nesterov pour améliorer le taux de convergence.

Algorithm 4 FISTA (Fast Iterative Soft-Thresholding Algorithm)

Require: X, y, λ, L, T

- 1: Initialiser $w_0 \in \mathbb{R}^d$, $z_0 = w_0$, $t_0 = 1$
 - 2: Choisir $\alpha = \frac{1}{L}$
 - 3: **for** $k = 0, 1, \dots, T - 1$ **do**
 - 4: Calcul du gradient en z_k : $g_k = \frac{1}{n}X^T(Xz_k - y)$
 - 5: Descente de gradient : $v_{k+1} = z_k - \alpha g_k$
 - 6: Soft-thresholding : $w_{k+1} = S_{\alpha\lambda}(v_{k+1})$
 - 7: Mise à jour du paramètre d'accélération : $t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$
 - 8: Combinaison momentum : $z_{k+1} = w_{k+1} + \frac{t_k - 1}{t_{k+1}}(w_{k+1} - w_k)$
 - 9: **end for**
 - 10: **return** w_T
-

Comparaison des taux de convergence

[Convergence d'ISTA] Pour une fonction f lisse avec gradient L -Lipschitz, ISTA satisfait :

$$f(w_k) + \lambda\|w_k\|_1 - (f(w^*) + \lambda\|w^*\|_1) \leq \frac{L\|w_0 - w^*\|^2}{2k}$$

Taux de convergence : $\mathcal{O}(1/k)$

[Convergence de FISTA] Pour la même fonction, FISTA satisfait :

$$f(w_k) + \lambda\|w_k\|_1 - (f(w^*) + \lambda\|w^*\|_1) \leq \frac{2L\|w_0 - w^*\|^2}{(k + 1)^2}$$

Taux de convergence : $\mathcal{O}(1/k^2)$

Comparaison pratique :

Pour atteindre une précision ϵ :

- ISTA nécessite $\mathcal{O}(1/\epsilon)$ itérations
- FISTA nécessite $\mathcal{O}(1/\sqrt{\epsilon})$ itérations

FISTA est asymptotiquement beaucoup plus rapide.

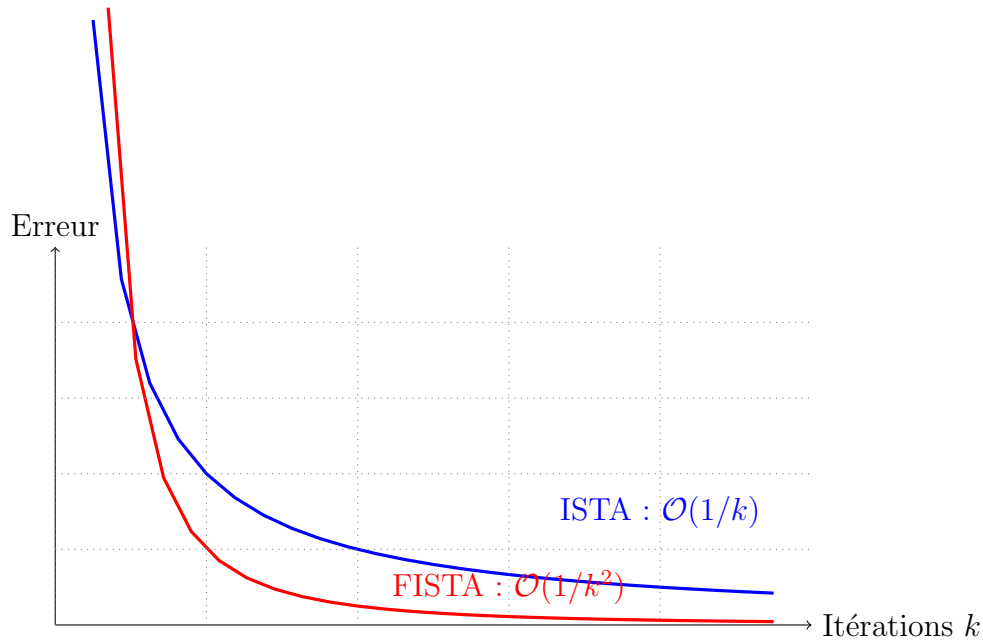


FIGURE 6 – Comparaison des taux de convergence : FISTA converge quadratiquement plus vite qu’ISTA

d) Sélection de variables

Principe

En faisant varier λ , on contrôle le niveau de parcimonie :

- $\lambda \rightarrow 0$: Pas de régularisation, tous les coefficients sont non nuls
- λ modéré : Certains coefficients sont mis à zéro
- $\lambda \rightarrow \infty$: Tous les coefficients sont nuls ($w = 0$)

Identification des mots significatifs

Pour la classification de documents (Reuters RCV1), chaque feature correspond à un mot. Les mots avec $w_j \neq 0$ sont les plus discriminants pour la classification.

Procédure :

1. Entraîner le modèle Lasso pour différentes valeurs de λ : $\lambda_1 > \lambda_2 > \dots > \lambda_K$
2. Pour chaque λ_i , compter le nombre de coefficients non nuls : $\|w(\lambda_i)\|_0$
3. Tracer le *regularization path* : w_j en fonction de λ
4. Identifier les mots qui restent actifs même pour λ élevé $\Rightarrow$ features les plus importantes

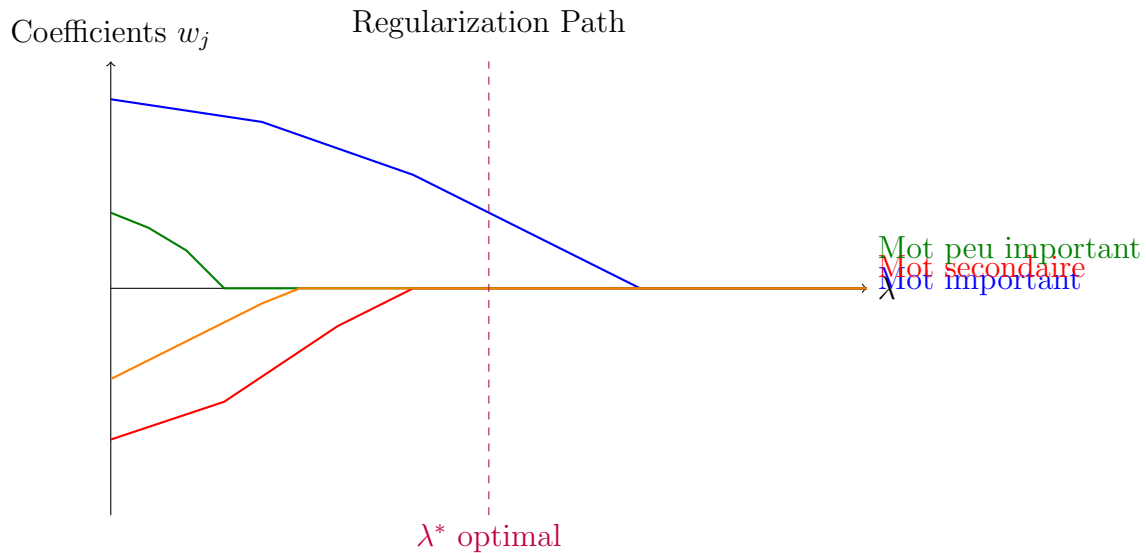


FIGURE 7 – Chemins de régularisation : les coefficients diminuent et s’annulent progressivement quand λ augmente

Critère de sélection :

On choisit typiquement λ par validation croisée pour maximiser la performance de classification tout en maintenant un modèle parcimonieux.

Conclusion

Ce projet a permis d’explorer les quatre piliers de l’optimisation en Machine Learning :

1. **Modélisation** : Formulation rigoureuse du problème, analyse de la forte convexité et lien avec l’unicité du minimum global via le Théorème 1.2.9.
2. **Méthodes déterministes** : Calcul analytique des gradients et de la Hessienne, détermination de la constante de Lipschitz via SVD et son impact sur la stabilité.
3. **Passage à l’échelle** : Preuve du caractère sans biais du SGD, analyse du bruit de gradient, implémentation de mini-batch et Adam, importance de la standardisation.
4. **Optimisation non lisse** : Analyse géométrique justifiant la parcimonie de L1, dérivation de l’opérateur proximal (soft-thresholding), algorithmes ISTA et FISTA avec analyse de convergence.